

Plan de Proyecto y Guía de Desarrollo: Aplicación web de estadísticas de Catán

Introducción

La idea principal de este proyecto es desarrollar una **aplicación web responsive** para registrar partidas del juego de mesa *Catán* entre grupos de amigos y obtener estadísticas útiles a partir de esos datos. La aplicación está pensada para usarse desde el **móvil durante las partidas** (minimizando la fricción al introducir datos) y posteriormente desde cualquier dispositivo para analizar las estadísticas en un panel tipo *dashboard*. El objetivo es crear una herramienta **sencilla pero poderosa** que permita a un grupo fijo de jugadores llevar la cuenta de sus partidas y visualizar métricas interesantes sobre su desempeño en Catán. Se buscará un diseño atractivo y fácil de usar, privilegiando la **usabilidad real por encima de funciones demasiado complejas**, dado que el desarrollador es junior y el tiempo de desarrollo para un MVP (Producto Mínimo Viable) se estima en ~3 meses ¹.

Objetivos del Proyecto

- **Registro rápido de partidas:** permitir ingresar los datos esenciales de cada partida de Catán de forma ágil, incluso mientras se está jugando, sin interrumpir el flujo del juego ².
- **Estadísticas claras y significativas:** generar métricas y visualizaciones estilo *dashboard* que reflejen el desempeño de los jugadores de forma realista (ej. porcentaje de victorias, puntos promedio, rachas), evitando estadísticas excesivamente complejas que resten usabilidad ³.
- **Orientado a grupos fijos:** la aplicación gira en torno a **grupos** de amigos que juegan entre sí. Cada usuario podrá crear o unirse a uno o varios grupos, y las estadísticas se calculan por grupo, manteniendo separadas las “peñas” de juego ⁴.
- **Soporte para expansiones de Catán:** se contemplará desde el inicio la expansión *Navegantes* (y en el futuro otras) como variantes de reglas configurables por partida. La arquitectura deberá facilitar agregar expansiones sin reestructuraciones mayores ⁵.
- **Multiplataforma web:** ofrecer una experiencia **responsive** accesible tanto en móviles (uso principal durante partidas) como en PC, sin necesidad de apps nativas (solo requiere un navegador web) ⁶.
- **Facilidad de desarrollo y mantenimiento:** usar tecnologías adecuadas (como FastAPI en backend) que permitan un desarrollo relativamente rápido y claro, apto para un desarrollador junior, sentando bases sólidas para mejoras futuras ⁷.

Alcance del Proyecto

En esta sección se definen las funcionalidades que **sí** se implementarán (alcance) en contraste con aquellas que deliberadamente quedan **fuera de alcance** en el MVP debido a limitaciones de tiempo o complejidad.

Funcionalidades Incluidas (En alcance)

- **Gestión de usuarios y autenticación:** registro de usuarios (crear cuenta con email/usuario y contraseña, almacenada de forma segura con hash) ⁸; inicio de sesión para usuarios

registrados con autenticación (se podría usar JWT o sesiones de FastAPI para proteger rutas privadas) ⁹ . Cada usuario tendrá un perfil básico (nombre mostrado, etc.), aunque la mayoría de datos estadísticos serán por grupo.

- **Gestión de grupos de juego:** un usuario autenticado puede **crear un nuevo grupo** (ej. "Amigos de la Universidad", "Familia Catán") asignándole nombre y descripción. También podrá **unirse a grupos existentes** mediante invitación: por simplicidad, se puede generar un **código o enlace de invitación** que el creador comparte con sus amigos, o permitir al creador añadir miembros por su nombre de usuario/correo ¹⁰ . Los grupos listarán sus miembros y se podrán ver en el panel principal del usuario (lista de grupos a los que pertenece) ¹¹ . No se implementará en MVP una gestión compleja de roles; asumiremos que todos los miembros pueden registrar partidas, aunque quizá solo el creador pueda invitar o remover miembros (control mínimo de administrador) ¹² .
- **Registro de partidas:** funcionalidad central para **ingresar una nueva partida** jugada dentro de un grupo determinado. Los datos a registrar incluyen:
 - Fecha de la partida (por defecto, la fecha actual).
 - Selección de jugadores participantes: se listan los miembros del grupo para marcar quién jugó. Normalmente 3-4 jugadores (Catán base), con posibilidad de hasta 5-6 si la expansión lo permite ¹³ (la interfaz debe impedir seleccionar un número inválido de jugadores).
 - Expansión o conjunto de reglas usado en la partida: por ejemplo *Base* (Catán clásico) o *Navegantes* (y en el futuro otras expansiones) ¹⁴ .
 - Indicar el **ganador** de la partida (por ejemplo, seleccionando uno de los jugadores participantes).
 - Puntos de victoria finales de *cada jugador*: un campo numérico junto a cada nombre para anotar su puntuación final (ejemplo: Alice 10, Bob 8, Carol 6) ¹⁵ .
 - Campos opcionales: duración aproximada de la partida (ej. "1h 30m") y notas o comentarios libres sobre la partida (por ejemplo, incidencias o recordatorios).
 - Al guardar la partida, el sistema validará que los datos estén completos y consistentes (ej. que el ganador esté entre los jugadores participantes, que la suma de puntos corresponda a las reglas esperadas, etc.).
- **Historial de partidas del grupo:** la pantalla principal de cada grupo mostrará la **lista cronológica de partidas** registradas con información resumida: fecha, jugadores participantes, expansión usada, ganador y puntuaciones ¹⁶ ¹⁷ . Por ejemplo, un ítem podría verse como: "12/01/2026 – 4 jugadores – Ganador: Alice (10 pts) – Expansión: Base". La lista estará ordenada de la más reciente a la más antigua, y cada partida podrá seleccionarse para ver detalles adicionales (p.ej. participantes y puntos completos, notas) ¹⁸ .
- **Estadísticas básicas por grupo:** cada grupo tendrá un panel de **estadísticas agregadas** a partir de sus partidas registradas. Inicialmente (MVP) se mostrarán métricas clave de forma numérica o en simples gráficas, tales como:
 - Número total de partidas jugadas por el grupo.
 - Número de victorias por jugador y su *win rate* (porcentaje de partidas ganadas).
 - Puntuación de victoria promedio por jugador (promedio de puntos finales).
 - Duración media de las partidas (si se registra la duración).
 - Partidas jugadas por mes (ej. cuántas partidas se juegan cada mes) ¹⁹ .
- **Perfil de jugador dentro del grupo:** además del panorama general, cada jugador (miembro) del grupo podrá ver sus **estadísticas personales** dentro de ese grupo:
 - Partidas jugadas y partidas ganadas.
 - Porcentaje de victorias (win rate personal).
 - Puntuación media obtenida en sus partidas.
 - Racha actual de victorias (cuántas victorias consecutivas lleva, si aplica).

- "Mejor compañero": con quién gana más a menudo en el mismo juego (por ejemplo, si en partidas de equipos o cooperativo, pero en Catán todos compiten; aquí podría interpretarse como contra quién suele quedar arriba juntos en el podio).
- "Némesis": qué oponente tiende a ganarle más veces (quién lo supera con más frecuencia).
- Historial de partidas individuales (lista filtrada de partidas del grupo en las que participó ese jugador, quizás destacando si ganó o su puntaje).
- **Estadísticas avanzadas (dashboard completo)**: una vez cubiertos los básicos, el objetivo es ofrecer visualizaciones gráficas y análisis más profundos para el grupo, por ejemplo:
- **Win rate por jugador en el grupo**: comparativa visual (gráfico de barras o de torta) del porcentaje de victorias de cada jugador ²⁰.
- **Puntuación media por jugador**: gráfica comparando el promedio de puntos finales que obtiene cada jugador.
- **Win rate por expansión**: desglosar las victorias según la expansión jugada. Esto permite ver si ciertos jugadores rinden mejor en Catán básico o en *Navegantes*, etc.
- **Evolución de partidas en el tiempo**: gráfico (línea temporal o barras) mostrando cuántas partidas se juegan en cada periodo (mes, año), para ver tendencias de actividad del grupo ¹⁹.
- **Matchups entre jugadores**: comparación directa entre dos jugadores específicos cuando compiten en la misma partida. Por ejemplo: "cuando Alice y Bob juegan juntos, Alice gana X% de esas partidas y Bob Y%".
- **Estadísticas "Catán puro"**: impacto de logros especiales en la victoria. Aquí se analiza cómo ciertos hitos de Catán afectan las probabilidades de ganar:
 - ¿Qué porcentaje de veces el ganador de la partida tenía la **Ruta Más Larga**? ¿Y el **Ejército más Grande**? (Datos de la comunidad sugieren que el ganador tiene la Ruta Más Larga aproximadamente en el 50% de las partidas, mientras que el Ejército más Grande solo en ~25-30% ²¹).
 - ¿Cuántas veces un jugador que obtuvo **ambos** (Ruta Más Larga y Ejército más Grande) terminó ganando? (Se estima que conseguir ambos bonus correlaciona con una probabilidad de victoria muy alta, cercana o superior al 90%, indicando que es casi seguro ganar si un jugador monopoliza ambos logros).
 - **Relación entre el orden de turno inicial y la victoria**: si se registra quién fue 1º, 2º, 3º, 4º en la partida, analizar si algún puesto de salida tiene ventaja estadística. Por ejemplo, ¿gana más a menudo el jugador que empieza primero? (En teoría Catán equilibra esto dando compensaciones al último jugador, pero la app podría revelar patrones del grupo).
 - **Estilos de juego**: identificar tendencias de los jugadores, por ejemplo jugadores de estilo constructor vs. desarrollador. Un jugador constructor podría ganar frecuentemente construyendo muchas carreteras y asentamientos (obteniendo Ruta Más Larga a menudo), mientras que otro jugador podría enfocarse en cartas de desarrollo y ejército (obteniendo Ejército más Grande). La app podría inferir esto de los datos (ej. cuántas victorias de un jugador incluyen Ruta Más Larga vs cuántas incluyen Ejército más Grande, etc.) ²² ²³.
- **Modo "En juego" (registro en tiempo real)**: se trata de un módulo opcional destinado a usarse durante el transcurso de una partida, para **registrar ciertos eventos o hitos clave en vivo** sin mucha carga para el usuario. En lugar de intentar anotar todo lo que ocurre (lo cual entorpecería el juego), este modo permite con unos pocos **toques rápidos en el móvil** registrar eventos importantes:
- **"Jugador X consigue la Ruta Más Larga"**: es decir, en el momento en que X construye suficientes carreteras para obtener la carta de Ruta Más Larga, se registra ese hito.
- **"Jugador Y consigue el Ejército más Grande"**: cuando Y juega su tercer caballero y reclama la carta de Ejército más Grande ²⁴.
- Botón de **+1 caballero** jugado para un jugador (contador de caballeros) ²⁵, llevando el seguimiento de quién va acumulando cartas de caballero.

- Actualización rápida de **puntos de victoria**: por ejemplo, si un jugador construye una ciudad durante la partida, se podría incrementar su puntaje en +1 en la app para llevar cuenta de puntos en tiempo real ²⁶.
- Cualquier otro evento clave que se desee incluir (por ejemplo, quién obtuvo cierta ficha especial en una expansión, etc., aunque el objetivo es enfocarse en *hitos mayores* y no en cada detalle menor).
- Con unos pocos inputs, la aplicación generará una **línea temporal (timeline) de la partida** que luego sirve para análisis más avanzado. Por ejemplo, se podrá revisar cómo evolucionó la partida: quién tomó primero la Ruta Más Larga y si logró retenerla hasta el final, en qué momento alguien alcanzó X puntos, cuándo se arrebató la Ruta Más Larga o el Ejército, etc. **Todo esto sin exigir anotar cada tirada de dado ni cada recurso**, priorizando solo lo necesario para enriquecer las estadísticas sin romper la usabilidad durante el juego ²⁷. *(Nota: Este modo es opcional; muchos grupos quizás prefieran no usarlo y solo registrar el resultado final, lo cual está bien. La app debe funcionar con ambos enfoques.)*
- **Soporte de expansiones (Reglas personalizables)**: desde el MVP se considerará soporte para al menos el Catán **Base** y la expansión **Navegantes**. Al crear una partida, el usuario podrá indicar qué conjunto de reglas o expansión está usando. Las expansiones se manejarán simplemente como una **configuración de reglas diferente**, no se pretende simular tableros ni mecánicas específicas en detalle, sino solo almacenar la información necesaria para estadísticas ¹⁴. Por ejemplo:
- *Navegantes* tiene como condición ganar con **13 puntos** en vez de 10 (según las reglas oficiales, la partida termina cuando un jugador alcanza 13 puntos) ²⁸ ²⁹. Por ello, al registrar una partida de *Navegantes* se debe permitir puntos hasta 13 (u otra forma de marcar que alguien llegó a 13).
- *Navegantes* introduce conceptos de explorar islas (puntos extras representados con fichas bonus). Podríamos registrar cuántos puntos de exploración extra obtuvo cada jugador, si resulta relevante para las estadísticas.
- La arquitectura estará preparada para añadir en el futuro expansiones como *Ciudades y Caballeros*, *Mercaderes y Bárbaros*, etc., sin “romper” el modelo de datos. Es decir, se podrán extender con nuevos campos o tipos de eventos las particularidades de cada expansión según sea necesario ³⁰. Por ejemplo, *Ciudades y Caballeros* no usa Ejército más Grande ni Ruta Más Larga, pero introduce otras métricas (caballeros activados, metropolis, etc.), las cuales podrían incorporarse mediante nuevos campos o tablas opcionales cuando llegue el momento.

Funcionalidades Fuera de Alcance (Exclusiones deliberadas)

Para mantener el proyecto abordable en tiempo y complejidad, se definen claramente qué **no** se implementará en el MVP:

- **Simulación del tablero ni tiradas de dados**: la app **no dibujará el tablero** de Catán ni representará la posición de carreteras, ciudades, hexágonos, etc. Tampoco se registrarán los resultados de cada tirada de dados ni los recursos obtenidos turno a turno. Ingresaremos solo datos *resumidos* por partida (puntos finales, quién ganó, logros como Ruta Más Larga, etc.), sin pretender digitalizar el juego físico en tiempo real ³¹.
- **Visión por computador / lectura de imágenes**: no se intentará **escanear el tablero con la cámara** para obtener datos automáticamente. **Todo input será manual** (aunque mantenido al mínimo y optimizado). Esto evita complejidad de implementar reconocimiento de imagen y posibles errores asociados ³².
- **Cálculos excesivamente complejos o irrelevantes**: se evita cualquier cálculo *demasiado “fiki”* que sacrifique usabilidad. Por ejemplo, no se intentará calcular probabilidades de victoria en vivo

ni métricas que no aporten valor claro a los jugadores ³³. El foco son estadísticas retrospectivas claras y divertidas, no un asistente de estrategia en tiempo real.

- **Juego en línea ni *matchmaking*:** la aplicación **no será una plataforma para jugar Catán en línea** contra otros, ni para emparejar jugadores desconocidos. Se asume que las partidas ocurren físicamente entre amigos y la app solo registra los resultados. No habrá funcionalidades de buscar partidas públicas ni salas de juego virtual ³⁴.
- **Aplicaciones nativas de escritorio o móvil:** no se desarrollarán apps nativas separadas. Nos concentraremos en una **web app responsive** que funcione bien en navegador móvil, en lugar de crear aplicaciones dedicadas para iOS, Android o PC. Esto simplifica el desarrollo y mantenimiento, dado el equipo de una sola persona ³⁵.

(Nota: Las exclusiones anteriores podrían reconsiderarse en el futuro del proyecto, pero quedan fuera del alcance del MVP de ~3 meses.)

Requisitos No Funcionales

Además de las funcionalidades, es importante establecer criterios de calidad y restricciones técnicas:

- **Usabilidad y UX:** la aplicación debe ser fácil de usar, con **interfaz simple e intuitiva**, especialmente en móvil. Ingresar datos durante una partida debe requerir pocos toques y ser lo menos disruptivo posible. Se priorizará la claridad (ej. botones grandes, formularios mínimos) para que los jugadores no pierdan mucho tiempo registrando información.
- **Responsive design:** la interfaz debe adaptarse a pantallas pequeñas de teléfono así como verse bien en pantallas de PC. Dado que el uso principal durante el juego será en móviles, se diseñará *mobile-first* y luego se asegurará que expanda adecuadamente a escritorio ³⁶.
- **Rendimiento:** con un volumen de datos relativamente bajo (grupos de amigos, quizás decenas o cientos de partidas a lo sumo), una implementación sencilla debería ser suficiente. Aun así, las consultas de estadísticas deben ser eficientes. Se puede cachear o pre-calcular algunas estadísticas si empiezan a crecer los datos. La app debe responder de forma ágil a las acciones del usuario (idealmente <500ms en operaciones habituales en local, quizá <1-2s en operaciones de cálculo de dashboard).
- **Confiabilidad y consistencia:** los datos de partidas deben almacenarse de forma segura y consistente. Debe haber transacciones al registrar partidas para que o se guarde todo (partida y sus jugadores/puntos) o nada, evitando datos incompletos si ocurre un fallo. Además, las estadísticas calculadas deben ser correctas; conviene probar con datos de ejemplo para verificar que, por ej., el cálculo de win rate, rachas, etc., es correcto ³⁷.
- **Seguridad:** se implementará autenticación para que solo usuarios autorizados accedan a los datos de sus grupos. Las contraseñas se almacenarán hasheadas (p.ej. usando bcrypt). Se protegerán las rutas sensibles (por ejemplo, no permitir editar partidas a quien no pertenece al grupo). Si se usa JWT, asegurar su expiración y almacenamiento seguro (HTTP-only if cookies). También considerar medidas básicas contra ataques comunes (rate limit en login para evitar fuerza bruta, validación de inputs para evitar SQL injection – aunque usando ORM esto se mitiga).
- **Portabilidad del entorno:** la app seguirá principios de *12-factor*. Por ejemplo, se podrá usar **SQLite en desarrollo** (para facilidad de configuración) pero **PostgreSQL en producción** para mayor robustez, con la posibilidad de cambiar de motor de base de datos sin esfuerzo gracias al ORM ³⁸. La configuración (credenciales DB, secretos) estará externalizada en variables de entorno para facilitar despliegue en distintos entornos.
- **Mantenibilidad y claridad de código:** dado que es un proyecto para un desarrollador junior, se estructurará el código de forma clara y modular, evitando complejidad innecesaria. Se utilizarán frameworks conocidos (FastAPI, SQLAlchemy) en lugar de código casero para cosas ya resueltas,

lo que reduce errores y facilita que otro desarrollador (o el mismo en el futuro) entienda el proyecto. Documentar el código y seguir convenciones (PEP8 en Python) será importante.

- **Escalabilidad futura:** aunque no esperamos miles de usuarios, la arquitectura debe permitir crecer en funcionalidades. Por ello separamos lógica en capas (API, servicios, datos) para poder modificar o ampliar partes específicas (p.ej. añadir una nueva estadística, o conectar un front-end diferente) sin reescribir todo. Usar un ORM también facilita escalar la base de datos a otro motor o a replicación si hiciera falta en un futuro.
- **Registro y monitoreo:** incorporar *logging* básico para registro de eventos y errores en el backend, lo cual ayuda a depurar durante desarrollo y monitorear en producción ³⁹. Por ejemplo, loggear cada petición importante, errores de servidor, etc. Si se despliega en la nube, considerar usar alguna herramienta de monitoreo sencillo o al menos revisar logs periódicamente.
- **Pruebas:** se recomienda implementar algunos **tests unitarios** o al menos pruebas manuales planificadas para las partes críticas. En particular, las funciones de análisis de datos (estadísticas) deben probarse con datos conocidos para verificar que calculan correctamente (ej. verificar que el win rate de un jugador con 2 victorias de 5 partidas es 40%). Dada la duración corta, no se hará *TDD* exhaustivo, pero sí pruebas mínimas para evitar errores lógicos en cálculos.

Arquitectura de la Solución

A continuación se describe la arquitectura técnica propuesta, incluyendo las tecnologías a utilizar, las capas de la aplicación y el modelo de datos principal.

Tecnologías a utilizar

- **Backend:** Python 3 (por su simplicidad y potencia) con el framework **FastAPI** para construir una API REST rápida y con buen soporte de documentación automatizada.
- **ORM y base de datos:** biblioteca **SQLAlchemy** para el mapeo objeto-relacional y manejo de la base de datos relacional. Base de datos SQLite durante desarrollo (cero configuración) y PostgreSQL en producción (robustez, concurrencia). Se usará también **Alembic** (herramienta de migraciones de SQLAlchemy) para versionar cambios en el esquema de la base de datos conforme el proyecto evoluciona ⁴⁰.
- **Validación de datos:** Pydantic (integrado con FastAPI) para definir esquemas de datos (modelos) y validar entradas/salidas de la API de forma fácil.
- **Análisis de datos:** librería **Pandas** para cálculos estadísticos sobre los datos de partidas (por ejemplo, para calcular promedios, agrupaciones, generar series temporales) de manera conveniente. Aunque muchas stats pueden obtenerse con consultas SQL, Pandas permitirá manipular los datos de forma más flexible para gráficos y métricas complejas.
- **Frontend:** inicialmente se usará renderizado de páginas HTML en el servidor con **Jinja2** (plantillas) integrado en FastAPI. Esto permite construir la interfaz (formularios, listas, tablas, dashboard básico) sin tener que desarrollar una aplicación front-end separada, reduciendo la complejidad para el MVP ⁴¹. Se complementará con **HTML5/CSS3** y posiblemente un framework CSS sencillo (*Bootstrap* o similar) para lograr un diseño responsive rápidamente. Puede añadirse un poco de **JavaScript** puro o librerías ligeras para gráficos (por ejemplo, Chart.js) a fin de hacer el dashboard interactivo, pero sin embarcarse en un framework JS pesado en esta etapa.
- **Control de versiones y despliegue:** se usará **Git** para llevar control del código. Para despliegue, se puede inicialmente utilizar un servicio en la nube sencillo (por ejemplo, Heroku, Railway, Deta, etc. que soporten FastAPI) o un VPS. Se considerará el uso de **Docker** para encapsular la aplicación y sus dependencias, facilitando el despliegue consistente en distintos entornos ⁴², aunque esto podría introducirse en fases posteriores dado que un desarrollador junior podría

primero hacer un despliegue manual sencillo y luego migrar a Docker cuando esté más familiarizado.

Arquitectura por capas

La solución seguirá una arquitectura típica de una aplicación web Python con separación de responsabilidades en capas:

- **Capa de Datos (Base de Datos):** se utilizará una base de datos relacional para almacenar todas las entidades principales (usuarios, grupos, partidas, etc.) ⁴⁰. En desarrollo será SQLite (archivo local) por simplicidad, y en producción PostgreSQL para mayor capacidad. El acceso a la base se hará exclusivamente a través del ORM (SQLAlchemy), evitando consultas SQL crudas en el código de negocio.
- **Modelo ORM (Modelos de datos):** se definirán *modelos* SQLAlchemy que mapean las tablas de la base de datos a clases Python. Por ejemplo, una clase `User` mapeará a la tabla `users`, `Match` a `matches`, etc. Estas clases incluirán las relaciones entre entidades (e.g. un `Match` tendrá una relación muchos-a-muchos con `User` a través de `MatchPlayer`) ⁴³. El ORM nos permite manipular registros como objetos Python y delegar a SQLAlchemy la traducción a SQL, lo que hace el código más limpio y agnóstico del motor de base de datos. SQLAlchemy es una librería madura y flexible para BD relacionales, lo que facilitará incluso consultas complejas o join automáticos, además de integrarse bien con Pandas si exportamos datos a data frames ⁴⁴.
- **Capa de Lógica de Negocio/Servicios:** encima del ORM, conviene crear funciones de **servicio** (o una capa *CRUD*) que encapsulen operaciones comunes. Por ejemplo, una función `create_match(datos)` que se encargue de crear una partida nueva junto con sus entradas en `MatchPlayer`, o `get_group_stats(grupo_id)` que consulte las partidas de un grupo y calcule las estadísticas agregadas (quizá usando Pandas para ciertas métricas) ⁴⁵. Separar esta capa permite mantener la lógica de negocio aislada de los controladores web, facilitando pruebas unitarias y futuras modificaciones (por ejemplo, si cambia la forma de calcular una estadística, se toca en el servicio pero las rutas de API siguen igual).
- **Capa API (Web API REST):** la capa superior expone una API HTTP usando FastAPI. Se organizarán las rutas en **routers** o controladores por recurso, similar a los *Blueprints* de Flask ⁴⁶. Ejemplos: un router `users` para rutas de usuarios (`/users/register`, `/users/login`...), otro `groups` para rutas de grupos (`/groups/{id}/...`), otro `matches` para partidas, y quizás uno para `stats`. Cada endpoint recibirá peticiones, validará datos de entrada con modelos Pydantic, llamará a la capa de servicios correspondiente y retornará la respuesta adecuada (normalmente JSON para la API). FastAPI automáticamente generará una documentación interactiva (Swagger UI) que será útil para probar la API durante el desarrollo ⁴⁷. *Nota:* Además de JSON, FastAPI permite devolver plantillas HTML renderizadas (usando `Jinja2Templates`), lo que aprovecharemos para las vistas web (ver siguiente punto).
- **Capa de Presentación (Frontend):** para el MVP, en lugar de construir un frontend separado en React/Vue, optaremos por **renderizar HTML en el backend** utilizando plantillas Jinja2 integradas con FastAPI ⁴¹. Por ejemplo, la ruta `/groups/{id}` podría devolver una plantilla que muestra el listado de partidas y estadísticas básicas del grupo, tomando los datos de la base. Esto simplifica el desarrollo para un junior, evitando la necesidad de manejar CORS, estados en cliente, etc., ya que el servidor entrega páginas completas. En un futuro, gracias a que la API REST existe, se podría desarrollar un cliente más dinámico (una SPA) o mejorar la interactividad con JavaScript sin recargar la página para ciertas acciones. Pero inicialmente, serviremos páginas tradicionales con algunos toques de JS solo si hace falta (por ejemplo, para gráficos interactivos usando los datos provistos por la API).

Esta arquitectura por capas asegura una separación de responsabilidades y una base sólida para extender la aplicación. Por ejemplo, podríamos reemplazar la capa de presentación por una app móvil consumiendo la API, sin alterar la lógica de negocio.

Modelo de Datos (Entidades principales)

A continuación se detallan las principales entidades (modelos de datos) que manejará la aplicación y cómo se relacionan entre sí:

- **User (Usuario):** representa a un usuario registrado en la aplicación (quien a su vez es un jugador en las partidas). Campos principales: `id`, `username` (o email), `password_hash` (contraseña encriptada), quizás `display_name` y avatar (opcional). Un usuario puede pertenecer a varios grupos. Esta entidad sirve para autenticación y para asociar resultados de partidas a identidades de jugador.
- **Group (Grupo):** representa un grupo fijo de juego. Campos: `id`, `name` (nombre del grupo), `description` (descripción opcional), `creator_id` (referencia al User que creó el grupo). Relaciones: un grupo tiene **muchos miembros** (usuarios) a través de la relación intermedia GroupMember, y tiene **muchas partidas** asociadas (todas las jugadas en ese grupo) ⁴⁸.
- **GroupMember:** tabla intermedia para la relación *muchos a muchos* entre User y Group (un usuario puede estar en varios grupos, un grupo tiene varios usuarios). Campos: `user_id`, `group_id` conformando la clave compuesta. Puede incluir atributos adicionales como `join_date` (fecha de ingreso al grupo) o `role` (rol del miembro, ej. *admin* o *miembro* regular). Esta entidad nos permite saber qué usuarios pertenecen a cada grupo y con qué rol/permisos ⁴⁹.
- **Match (Partida):** representa una partida de Catán jugada en un grupo. Campos: `id`, `group_id` (FK al grupo donde se jugó), `date` (fecha/hora de la partida), `ruleset_id` (FK al conjunto de reglas o expansión usada), `winner_id` (FK al User ganador, que debe ser uno de los jugadores participantes), `duration` (duración opcional estimada, en minutos por ejemplo), `notes` (texto libre con comentarios) ⁵⁰. Relaciones: una partida pertenece a un **único grupo**; y una partida tiene **múltiples jugadores** participando a través de la relación MatchPlayer.
- **MatchPlayer:** representa la participación de un **jugador (usuario) en una partida** específica. Es una tabla intermedia para la relación *muchos a muchos* entre Match y User, pero además contiene datos adicionales de esa participación. Campos: `match_id`, `user_id` (FKs que juntos identifican unívocamente una entrada), y campos extra como `victory_points` (puntos de victoria que ese jugador obtuvo en la partida) ⁵¹. También podríamos incluir aquí **banderas** o indicadores de logros en la partida: por ejemplo `longest_road = True/False` y `largest_army = True/False` para marcar si ese jugador obtuvo la Ruta Más Larga o el Ejército más Grande en esa partida ⁵². (Alternativamente, esos indicadores podrían estar en la tabla Match, pero guardarlos en MatchPlayer permite más flexibilidad, especialmente si en el futuro nos interesa quién *terminó* con esos logros frente a quién los tuvo temporalmente). Con MatchPlayer podemos consultar fácilmente, por ejemplo, los puntos de cada jugador en la partida, o identificar quién fue el ganador (coincidirá con `winner_id`).
- **MatchEvent (Evento de Partida):** modela un evento o hito ocurrido durante una partida, para soportar el **timeline** en caso de usar el modo en juego. Campos: `id`, `match_id` (FK de la partida), `timestamp` (marca de tiempo o secuencia), `event_type` (tipo de evento, por ej.: "LONGEST_ROAD", "LARGEST_ARMY", "KNIGHT_PLAYED", "BUILT_CITY", etc.), `player_id` (FK al jugador involucrado en el evento, si aplica), `value` (valor numérico opcional asociado, por ejemplo podría guardar **longitud de la Ruta Más Larga** en el momento del evento, o cuántos caballeros jugados lleva cuando obtuvo Ejército más Grande, etc.) ⁵³. Cada registro de MatchEvent indica que en cierto momento de la partida ocurrió algo

destacable, que luego podemos usar para análisis (p.ej. "al turno 50, Alice tomó la Ruta Más Larga con longitud 6"). Si no se usa el modo en juego, esta tabla permanecerá vacía o no utilizada.

- **Ruleset (Conjunto de Reglas/Expansión):** define la configuración de reglas para una partida según la variante de Catán. Campos: `id`, `name` (nombre de la variante, ej. "Base", "Navegantes"), `victory_points_required` (puntos de victoria necesarios para ganar: 10 en base, 13 en Navegantes ⁵⁴ ²⁹, etc.), `max_players` (número máximo de jugadores permitido: 4 para base, 4 para Navegantes estándar, 6 si se aplica la expansión de 5-6 jugadores), y quizás una serie de flags booleanos para indicar qué mecánicas especiales están presentes: `has_longest_road`, `has_largest_army` (en Catán base y Navegantes serían True ambas), o flags para expansiones futuras (p.ej. *Ciudades y Caballeros* no tiene Ruta Más Larga ni Ejército más Grande, así que esos serían False pero tendría otros flags como `has_metropolis = True`, etc.) ⁵⁵. De esta forma, el sistema sabe interpretar los datos según la expansión: por ejemplo, en una partida con ruleset "Ciudades y Caballeros" ignoraría estadísticas de Ruta Más Larga porque no aplican, pero podría habilitar campos como "Knight Strength" u otros.
- **PlayerProfile (Perfil de Jugador en grupo):** *opcional*. Conceptualmente representa un **resumen de estadísticas acumuladas** de un jugador en un grupo. Podría ser una entidad física en la BD con campos como `user_id`, `group_id` y contadores acumulativos: `games_played`, `games_won`, `total_victory_points`, `longest_road_won_count` (veces que ganó teniendo Ruta Más Larga), etc. ⁵⁶. Sin embargo, mantener estos datos actualizados en tiempo real añade complejidad (habría que recalcular o actualizar tras cada partida). Dado que con consultas SQL o cargando los datos en Pandas se pueden calcular estas estadísticas al vuelo, es probable que inicialmente **no implementemos PlayerProfile como tabla persistente**, sino que calculemos las stats dinámicamente cuando se piden. Podríamos cachear algunos resultados si fuera necesario. Quizás solo almacenaríamos algunas cosas difíciles de calcular incrementalmente, como la racha actual de victorias, para no tener que derivarla cada vez (aunque también es calculable). En resumen, el concepto de "perfil de jugador" existirá en la interfaz (mostrar sus stats), pero esos datos se derivarán de las partidas registradas en lugar de mantenerse en una tabla separada en MVP ⁵⁷.

Relaciones principales entre entidades: un **Usuario** puede pertenecer a muchos **Grupos** (y un grupo tiene varios usuarios) vía la tabla **GroupMember**. Un **Grupo** tiene muchas **Partidas (Match)**, pero cada partida pertenece a un solo grupo. Un **Match** tiene varios **Usuarios** participantes vía **MatchPlayer** (y un usuario puede aparecer en múltiples partidas a lo largo del tiempo). Cada partida se asocia a un **Ruleset** (expansión) que define las reglas aplicadas. Un **MatchEvent** pertenece a un Match específico y potencialmente referencia a un usuario (según el tipo de evento). Estas relaciones permiten navegar y consultar datos fácilmente; por ejemplo, dada una partida, se puede listar sus jugadores y sus puntajes a través de MatchPlayer, o dada una usuario y un grupo, se puede obtener todas sus partidas en ese grupo cruzando User -> MatchPlayer -> Match.

Metodología de Desarrollo

Dado que el desarrollo será realizado por **una sola persona** y el objetivo es lograr un MVP en ~3 meses, adoptaremos una metodología **híbrida** entre planificación anticipada e iteración ágil:

- Inicialmente se realiza esta **fase de planificación y diseño** del proyecto (definiendo requisitos, alcance, arquitectura). Esto proporciona un mapa claro a seguir.
- Luego, la implementación se llevará a cabo en **fases iterativas** (ver apartado de Fases del Proyecto). Cada fase actúa similar a un *sprint* ágil enfocado en un conjunto de funcionalidades relacionadas, al cabo del cual se obtiene una versión funcional (aunque sea básica) de la aplicación.

- Se prioriza entregar primero un **MVP utilizable** con lo esencial (registro de partidas y stats básicas) y después añadir características avanzadas gradualmente. Esta estrategia garantiza tener algo de valor tangible lo antes posible, y permite recibir feedback temprano (por ejemplo, probar la app en partidas reales del grupo de amigos e identificar mejoras).
- En cada iteración, el ciclo será: desarrollar las nuevas funciones -> realizar pruebas (manuales y/o unitarias) -> corregir errores -> desplegar/entregar esa versión -> recopilar feedback (si lo hubiera). Luego se planifica la siguiente iteración.
- Aunque sea un solo desarrollador, es recomendable llevar una especie de **lista de tareas o Kanban** (por ejemplo con Trello, GitHub Projects o incluso un documento) para organizar el trabajo de cada fase y no olvidar detalles.
- Se aplicarán principios ágiles de flexibilidad: si durante el desarrollo se descubre que alguna funcionalidad es demasiado compleja o poco útil, se puede ajustar el alcance o posponerla para mantener el cronograma. Inversamente, si todo va bien y queda tiempo, se pueden incorporar algunas de las mejoras opcionales.
- **Control de versiones:** usando Git para gestionar el código, con commits frecuentes y significativos. Esto permite revertir si algo sale mal y llevar historial de cambios. Si es posible, etiquetar versiones al final de cada fase (v0.1, v0.2, etc.).
- **Comunicación y feedback:** aunque es un proyecto personal, es útil mostrar prototipos o explicarle el progreso a algún amigo jugador de Catán para obtener impresiones. Esto ayuda a refinar la usabilidad (por ejemplo, tal vez descubran que sería útil un cierto atajo en la UI).
- **Tiempo y dedicación:** en 3 meses (~12 semanas) para MVP, se deberá ser realista con las cargas de trabajo. Cada fase tendrá estimaciones, pero se debe reservar tiempo para pruebas, corrección de bugs e integración. Es mejor tener menos funcionalidades pero pulidas, que muchas a medio terminar.
- Al final de la fase MVP, se revisará el estado del proyecto: debe estar lo suficientemente completo para usarse cómodamente en las sesiones de juego del grupo. A partir de ahí, se pueden planificar nuevas iteraciones para las características opcionales o expansiones.

En resumen, se planifica de antemano *qué* hacer (como en cascada tradicional) pero se ejecutará *cómo* hacerlo de forma iterativa y adaptable (metodología ágil). Esto le da al desarrollador junior un camino claro pero con flexibilidad para aprender y ajustar sobre la marcha.

Fases del Proyecto (Roadmap de Implementación)

Se propone dividir el desarrollo en **fases** secuenciales, cada una con entregables funcionales. A continuación, se detalla cada fase con su contenido y objetivos:

Fase 1: MVP Básico – Autenticación, Grupos, Partidas y Estadísticas Iniciales

- **Duración estimada:** 4-6 semanas.
- **Descripción:** Esta fase construye la base del sistema, permitiendo ya usar la aplicación en su forma más simple.
- **Funcionalidades a desarrollar:**
 - **Registro e inicio de sesión de usuarios:** formulario de registro de cuenta y pantalla de login. Implementar almacenamiento seguro de contraseñas (hash + sal). Tras login, manejo de sesión o JWT para mantener al usuario autenticado.
 - **Creación y gestión de grupos:** posibilidad de crear un nuevo grupo indicando nombre (y descripción opcional). Mostrar en el *dashboard* principal la lista de grupos del usuario. Implementar la unión a grupos mediante un mecanismo sencillo (p.ej. introducir código de invitación proporcionado por el creador, o que el creador agregue miembros).

- **Registro de partidas:** formulario dentro de un grupo para crear una nueva partida, con campos: fecha (por defecto hoy), jugadores participantes (checkbox o multiselect de miembros del grupo), selección de expansión (Base o Navegantes), ganador, puntos de cada jugador, duración opcional, notas. Al enviar, guardar la partida en la base de datos junto con sus MatchPlayer asociados (puntos por jugador).
- **Listado de partidas:** en la página de cada grupo, debajo del formulario o en una vista separada, listar las partidas registradas con info resumida. Asegurar que están ordenadas por fecha descendente. Permitir quizás hacer clic en una partida para ver detalles (que podría ser simplemente expandir para mostrar todos los puntos y notas).
- **Estadísticas básicas:** calcular a nivel de grupo: total de partidas, victorias por jugador (número y %), puntos promedio por jugador, etc., y mostrarlas en un pequeño panel. En Fase 1 esto puede ser estático o recalculado cada vez que se muestra la página (dado el bajo volumen de datos). Se puede mostrar, por ejemplo, una tabla o una lista de "Victorias:" con cada jugador y su porcentaje.
- **Interfaz mínima pero funcional:** utilizar plantillas HTML sencillas con algo de estilo (usar Bootstrap CSS por CDN, por ejemplo) para que la aplicación sea presentable. Optimizar para móvil (layouts verticales, menús simples).
- **Entrega esperada:** al finalizar Fase 1, la aplicación permitirá a un usuario registrarse, crear un grupo, agregar a sus amigos, **registrar partidas y ver estadísticas básicas**. Ya en este punto el grupo podría empezar a usar la app en sus sesiones de juego para llevar el marcador. Este es el **MVP** inicial utilizable.

Fase 2: Perfiles de Jugador y Dashboard de Estadísticas Completo

- **Duración estimada:** 3-4 semanas.
- **Descripción:** En esta fase se enriquece la aplicación con más vistas de estadísticas y datos individuales.
- **Funcionalidades a desarrollar:**
- **Perfil de jugador en grupo:** crear una vista o sección donde, al seleccionar un jugador (miembro) del grupo, se muestren sus estadísticas personales: partidas jugadas, ganadas, porcentaje, puntuación media, racha actual, etc., calculadas a partir de los datos. Incluir también datos de "mejor compañero" o "némesis" si es factible: por ejemplo, quién de los otros jugadores aparece más veces en sus partidas ganadas (ese podría ser su compañero favorable), o quién lo ha vencido más veces en partidas donde ambos jugaron (némesis).
- **Gráficos y visualización de datos:** integrar una librería de gráficos (p. ej. Chart.js) para presentar las estadísticas del grupo de forma visual en un **dashboard**. Por ejemplo:
 - Gráfico de barras o pastel para distribución de victorias por jugador.
 - Gráfico de líneas para partidas jugadas a lo largo del tiempo (mes a mes).
 - Quizá un gráfico de barras agrupadas para comparar puntos promedio.
- **Filtros de estadísticas:** añadir controles para filtrar o segmentar las estadísticas, por ejemplo:
 - Filtrar por expansión: ver las stats considerando solo partidas Base o solo Navegantes.
 - Filtrar por rango de fechas (último año, último mes, etc.) si el dataset crece.
 - Seleccionar dos jugadores específicos para comparar su *head-to-head* (esto podría ser simplemente mostrando su matchup stats).
- **Cálculo de estadísticas avanzadas:** implementar en backend las funciones para calcular las métricas avanzadas descritas en requisitos:
 - Calcular automáticamente las tasas de victoria con Ruta Más Larga, con Ejército más Grande, etc., usando los campos que registramos (de MatchPlayer booleans, por ejemplo). Mostrar en el dashboard algo como "El ganador tuvo Ruta Más Larga en X% de partidas del grupo" ²¹.

- Calcular y mostrar las estadísticas de matchups: probablemente una matriz o una pequeña tabla cuando se seleccione un enfrentamiento específico, ej: "Alice vs Bob: Alice ganó 5 de 8 partidas (62.5%)".
- Mostrar si es posible la distribución de puestos (1er, 2do, 3ro, 4to) que ocupa cada jugador, si decidimos guardar esa info. (Esto es opcional y podría diferirse).
- **Refinamiento de UI/UX:** mejorar la presentación general: por ejemplo, añadir enlaces para navegar (de la lista de partidas a detalle, de los nombres de jugadores al perfil, etc.), asegurarse de que en móvil los gráficos se puedan deslizar o ver bien, utilizar colores distintivos para cada jugador en las gráficas, etc. Hacer la aplicación más **pulida visualmente** ahora que las funcionalidades principales están.
- **Gestión de datos:** posiblemente incorporar la posibilidad de **editar o borrar partidas** (por si se ingresó algo mal). Al menos, un botón para borrar una partida podría implementarse con confirmación, que elimine la partida y actualice stats.
- **Entrega esperada:** al final de Fase 2, la aplicación ofrecerá un **dashboard completo de estadísticas** y perfiles individuales. Los usuarios del grupo podrán no solo ingresar partidas sino también explorar en profundidad sus datos históricos de juego de manera visual e interactiva. La experiencia de usuario será mucho más rica.

Fase 3: Modo "En juego" y Registro de Eventos en Tiempo Real

- **Duración estimada:** ~3 semanas.
- **Descripción:** Esta fase agrega el módulo opcional para registro en tiempo real durante las partidas, y aprovecha esa información para enriquecer análisis.
- **Funcionalidades a desarrollar:**
- **Interfaz de Modo en Juego:** dentro de la vista de un grupo, añadir una sección "Modo En Juego" donde se pueda iniciar el seguimiento de una partida en curso. Por ejemplo, se seleccionan los jugadores que van a jugar (como al registrar partida, pero aquí para iniciar el tracking). Esto crea un objeto Match en estado "en juego" y una pantalla específica para anotar eventos.
- **Registro de eventos clave:** implementar botones o controles de rápido acceso para los eventos definidos:
 - Botón "Ruta Más Larga" para asignar la Ruta Más Larga al jugador X (y quizá automáticamente quitarla de quien la tenía antes, si actualizamos ese estado).
 - Botón "Ejército más Grande" para asignar el Ejército más Grande.
 - Botón "+1 Caballero (Jugador X)" para registrar que X jugó un caballero (esto ayudaría a determinar cuándo alguien logra 3 caballeros).
 - Botón "+1 Punto (Jugador X)" para incrementar la puntuación de X (p.ej. si construye ciudad o asentamiento).
 - (Estos son ejemplos; en la implementación podríamos tener un pequeño menú por jugador con opciones, etc.)
- **Almacenamiento en timeline:** cada acción presionada genera un **MatchEvent** en la base de datos con timestamp y detalles. Mantener en la UI una lista visible de eventos registrados durante esa partida, a modo de log.
- **Finalizar partida en juego:** cuando la partida termina (alguien llega a 10/13 puntos), permitir finalizar y guardar definitivamente la partida. En ese momento, pedir confirmar los puntos finales de cada jugador (que tal vez ya fueron llevados cuenta por la app) y quién ganó. Esto consolidará la partida igual que en Fase 1 pero con los eventos guardados enlazados.
- **Visualización de timeline:** en la vista de detalles de una partida (o en una sección aparte), mostrar la línea temporal de eventos registrados: por ejemplo, "*Minuto 20: Alice obtiene Ruta Más Larga (longitud 5)*", "*Minuto 35: Bob juega su tercer caballero y obtiene Ejército más Grande*", etc. Esto se puede mostrar como una lista ordenada por tiempo.

- **Análisis usando eventos:** aprovechar los datos extra para estadísticas más finas, como:
 - Gráficas de progreso de puntos por turno (quién iba liderando a lo largo de la partida).
 - Indicadores de eficiencia: ej. "Jugador X suele liderar temprano pero no siempre gana", etc., aunque esto es complejo de automatizar, al menos los datos estarán y se pueden observar manualmente.
 - Datos sobre **disputas de Ruta Más Larga/Ejército**: cuántas veces cambian de manos, quién suele arrebatárselos a quién.
- **Optimización de usabilidad:** dado que este modo se usa durante la partida, haremos pruebas internas para asegurarnos de que es **rápido de usar**. Posiblemente incorporar sonidos o vibración (si es web no mucho, pero al menos feedback visual inmediato) para confirmar un evento registrado, de modo que el encargado de la app en la partida pase poco tiempo con el móvil.
- **Entrega esperada:** tras Fase 3, los grupos interesados en estadísticas avanzadas podrán **registrar partidas en tiempo real**. Esto añade valor para los jugadores más dedicados, generando una capa de datos rica. Sin embargo, la app seguirá permitiendo registrar solo el resultado final si así lo prefieren, manteniendo ambos usos. Técnicamente, la aplicación ahora tiene mayor complejidad de datos (eventos secuenciales) pero ofrece características únicas (timeline, análisis dinámico).

Fase 4: Integración de la expansión *Navegantes* y soporte ampliado de jugadores

- **Duración estimada:** 2-3 semanas.
- **Descripción:** En esta fase final del roadmap inicial, se pulen aspectos de expansiones y escalabilidad de número de jugadores, preparando el terreno para futuras expansiones.
- **Funcionalidades a desarrollar:**
- **Soporte completo de Navegantes:** asegurar que cuando se registra una partida con ruleset "Navegantes":
 - Se pueda indicar correctamente que la partida es a 13 puntos (y la interfaz refleje eso, por ejemplo cambiando "Puntos de victoria (max 10)" a "(max 13)").
 - Incluir campo(s) adicionales si decide capturarse información propia de Navegantes. Por ejemplo, podríamos añadir en Match un campo `extra_points` o en MatchPlayer `exploration_points` para indicar cuántos puntos de misión/islas consiguió cada jugador. Así, en stats podríamos eventualmente mostrar "Jugador X obtuvo Y puntos por exploración de islas".
 - Manejar la Ruta Más Larga en Navegantes: en esta expansión, la **Ruta Más Larga** cuenta carreteras y *barcos* conectados ⁵⁸. Esto no afecta cómo lo registramos (seguirá habiendo un jugador con Ruta Más Larga final), pero es un detalle de reglas a tener en cuenta (posiblemente en las notas o en cómo se explica al usuario).
 - Si surgen otras pequeñas reglas (p.ej. el uso del Pirata), pueden ignorarse en lo que respecta a registro, ya que no impactan directamente las estadísticas finales estándar.
- **Soporte de 5-6 jugadores:** si bien no es exactamente parte de Navegantes, es común que los grupos que juegan expansiones tengan también la **extensión de 5-6 jugadores**. Adaptaremos la aplicación para permitir hasta 6 jugadores en una partida **si el ruleset seleccionado lo permite**. Por ejemplo, definir `max_players` en Ruleset: Base=4, Navegantes=4 (aunque existe Navegantes 5-6), Base+5-6=6, Navegantes+5-6=6. Para simplificar, podríamos asumir que si un grupo tiene más de 4 miembros y selecciona "Base (5-6 jugadores)" o "Navegantes (5-6)", la UI permita elegir 5 o 6 jugadores. Esto implicará validar que el número de jugadores seleccionados no exceda lo que dicta el ruleset ⁵⁵.
- **Estadísticas por expansión:** agregar en el dashboard la posibilidad de ver las estadísticas **filtradas por tipo de juego**. Ya en Fase 2 se mencionó filtro por expansión; aquí lo consolidamos: por ejemplo, en el dashboard se puede tener una pestaña o sección "Estadísticas

Catán Base" y "Estadísticas Navegantes". Allí se ve, por ejemplo, que cierto jugador tiene 5 victorias de 10 en base (50%) pero 3 de 3 en Navegantes (100%), etc. Esto es útil para grupos que alternan modos de juego.

- **Pruebas y ajustes finales:** probar a fondo con un escenario de Navegantes: registrar partidas con 4 y con 5-6 jugadores, verificar que todos los cálculos (ej. porcentaje de victorias) siguen funcionando con distinto número de jugadores y diferentes puntos. Corregir cualquier bug en agregaciones (por ejemplo, cuando un jugador no participa en algunas partidas, sus promedios deben calcularse adecuadamente sin contarlas, etc.).
- **Entrega esperada:** finalizada Fase 4, la aplicación soportará de forma **robusta Catán Base y Navegantes**, con hasta 6 jugadores por partida. La estadística por expansión permitirá a los jugadores ver su desempeño en cada modalidad. La arquitectura quedará demostrada como flexible, al haber incorporado una expansión sin mayores complicaciones. En este punto, el MVP inicial se convierte en una aplicación más completa y lista para eventualmente extender a otras expansiones.

Extras Opcionales a Futuro

Si el tiempo lo permite, o para versiones posteriores del proyecto, se consideran algunas características adicionales que serían interesantes pero no prioritarias en el MVP:

- **Sistema de Ranking (ELO) simple:** implementar un cálculo de rating tipo ELO u otro sistema para rankear a los jugadores del grupo según sus resultados. Cada partida ajustaría los puntajes de los jugadores (ganar sumaría puntos, perder restaría en algún grado). Esto daría una idea de quién está *jugando mejor* en términos de consistencia, más allá de solo win rate.
- **Página pública del grupo:** generar una página o URL pública (que no requiere login) donde se muestren estadísticas agregadas del grupo, sin detalles sensibles. Por ejemplo, un resumen con "Grupo X: Alice 40% victorias, Bob 35%, Carol 25%, partidas jugadas 20". Esto permitiría compartir el desempeño con otras personas de forma sencilla. Obviamente, sería opcional y controlado por la privacidad (el grupo podría optar por no hacer públicas sus stats).
- **Integración con otras expansiones:** una vez sentadas las bases, agregar expansiones como *Ciudades y Caballeros*, *Mercaderes y Bárbaros*, *Piratas y Exploradores*, etc. Cada una requeriría identificar qué nuevos datos aportar (Ciudades y Caballeros, por ejemplo, podría requerir llevar cuenta de puntos de metrópolis, progreso de comodidades, etc.). Gracias al diseño con Ruleset y MatchEvent, debería ser factible añadir estas variantes extendiendo el modelo de datos.
- **Analytics avanzados:** por ejemplo, análisis de lanzamiento de dados si en algún momento se decidiera registrar tiradas (se dijo que no en alcance, pero podría hacerse como curiosidad en versiones posteriores), o análisis de recursos (si alguien ingresa manualmente qué producción tuvo, aunque es poco probable), etc. También cosas como "*¿qué tan a menudo alguien gana desde atrás?*" (comeback analysis) usando el timeline de puntos.
- **Mejoras de UI/UX:** un rediseño más profesional una vez establecida la funcionalidad, quizás con ayuda de un diseñador. Incluyendo mejores visualizaciones, animaciones en el dashboard, etc. También soporte multilenguaje (por si los jugadores quisieran el app en inglés, etc.).
- **Aplicación móvil nativa:** si la web app tiene éxito dentro del círculo, eventualmente se podría considerar envolverla en un contenedor nativo (usando frameworks tipo React Native, Flutter, o simplemente WebView) para tener presencia en tiendas de apps, aunque siendo realmente la misma web app. Esto no agrega funcionalidad pero puede mejorar la accesibilidad para algunos usuarios.

Estas mejoras se abordarían después de completar las fases principales, evaluando costo-beneficio. Es importante no intentar implementarlas antes de tiempo para no desviarse del MVP.

Conclusión

En resumen, este documento de planificación presenta un camino claro y detallado para construir la **aplicación web de estadísticas de Catán** desde cero. Se han definido objetivos precisos, un alcance realista para un desarrollador junior, una arquitectura sólida y modular, así como un plan de implementación dividido en fases manejables. Siguiendo esta guía, en aproximadamente **3 meses** de trabajo focalizado se puede lograr un **Producto Mínimo Viable** que permita a un grupo de amigos registrar sus partidas y visualizar estadísticas básicas. A partir de ese MVP, el proyecto está estructurado para crecer de forma incremental, incorporando características más avanzadas (dashboard gráfico, modo en juego, expansiones) que enriquecerán la experiencia.

Con la combinación de las tecnologías elegidas (FastAPI, SQLAlchemy, etc.) y las buenas prácticas mencionadas (validación, pruebas, logging), el desarrollador podrá desarrollar con confianza, aprendiendo en el proceso pero evitando muchos escollos típicos gracias a la planificación previa. En última instancia, la aplicación resultante no solo será un logro técnico, sino también una herramienta divertida y útil para la comunidad de jugadores de Catán, brindándoles *insights* sobre sus partidas y haciendo el juego aún más disfrutable fuera de la mesa. ¡Manos a la obra y que comiencen las estadísticas! ²¹ ²⁹

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 22 23 24 25 26 27 28 30 31
32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 Plan de

proyecto – Aplicación web de estadísticas de Catán.pdf

file:///file_00000000c9d471f4bfed8c8b012e72fd

²¹ ⁵⁸ Armies and Roads (Or Maybe Not) | Developing Catan

<https://developingcatan.wordpress.com/2010/11/04/armies-and-roads-or-maybe-not/>

²⁹ catan.com

https://www.catan.com/sites/default/files/2021-06/catan-seafarers_2021_rule_book_201201.pdf